

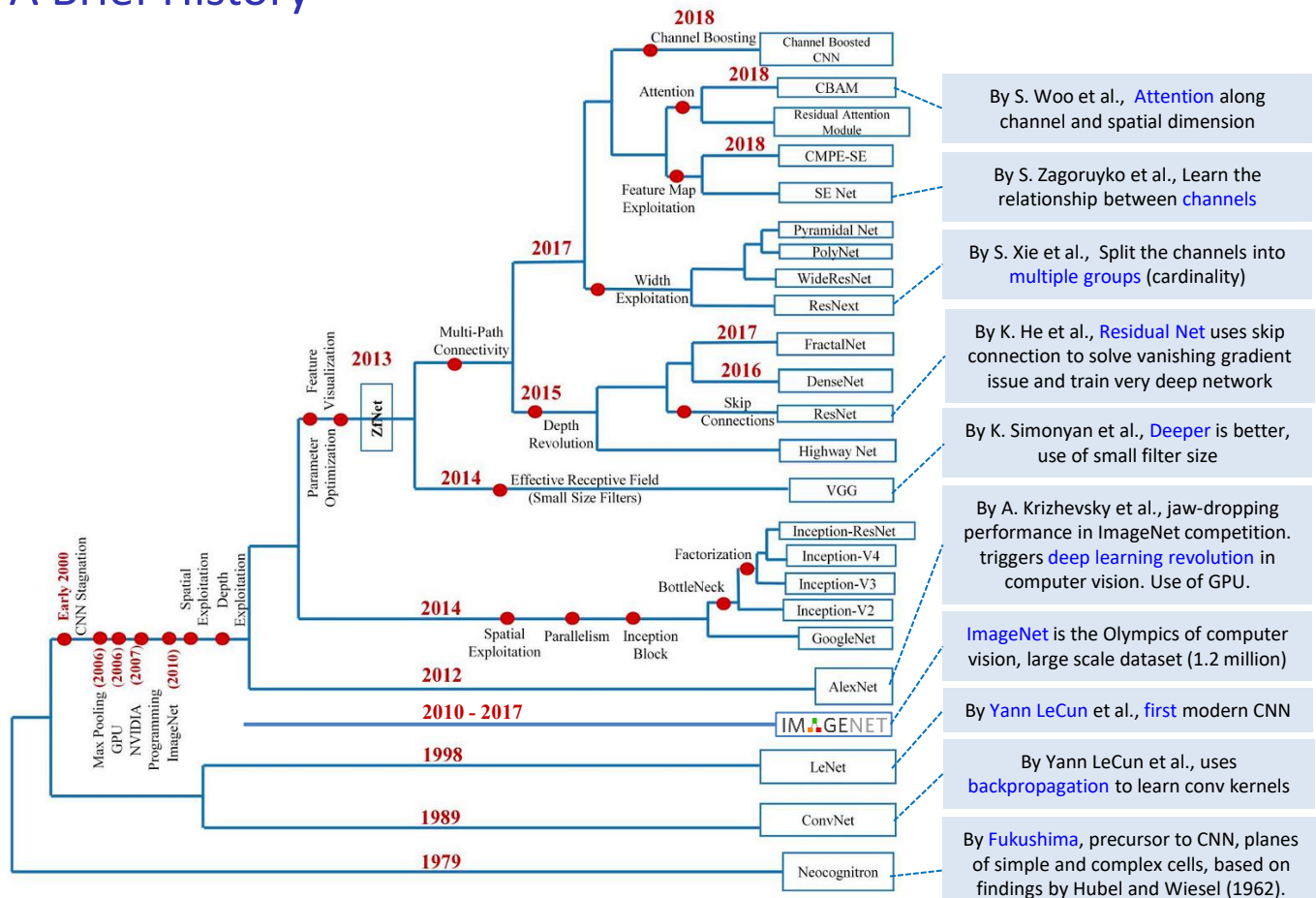
L04: Convolutional Neural Network

FICT, UTAR

Table of Contents

1. Background
2. Convolutional Layer
3. Configuring a Convolutional Layer
4. Number of Parameters and Computations
5. Pooling Layer
6. Variants of Convolutional Layer

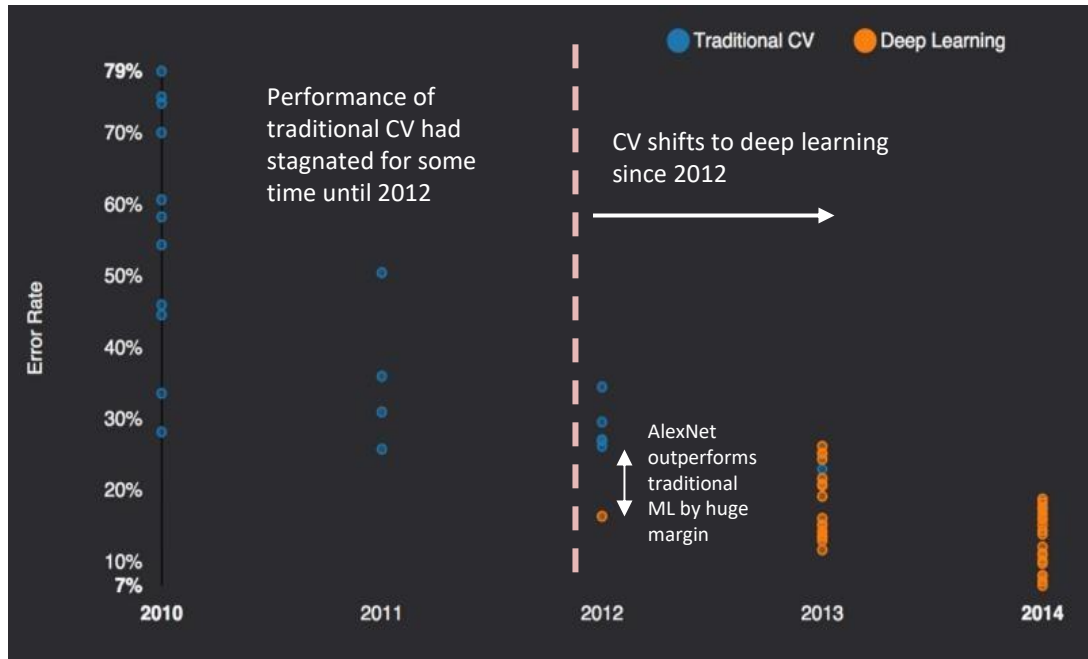
A Brief History



Reference: <https://towardsdatascience.com/top-10-cnn-architectures-every-machine-learning-engineer-should-know-68e2b0e07201>

Brief History: Deep learning revolution in computer vision

- AlexNet (2012) triggers the deep learning in computer vision.



Problems with Linear Layer

- Full connectivity is wasteful. For image data, standard neural network does not scale well.



Shape: (3,32,32)



Shape: (3,1000,1000)

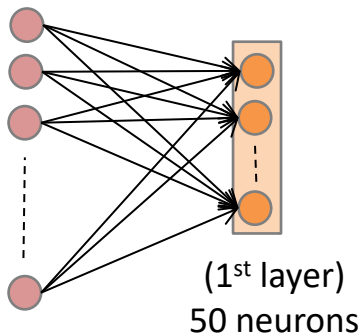


Image shape = (3, 32, 32):

Shape of $\mathbf{W}$ = (3*32*32, 50), b : (50,)

$$\begin{aligned}\text{\#params} &= (3*32*32)*50 + 50 \\ &= 153,650\end{aligned}$$

Image shape = (3, 1000, 1000):

Shape of $\mathbf{W}$: (3*1000*1000, 50), b : (50,)

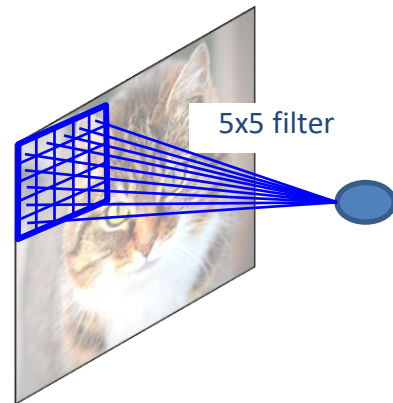
$$\begin{aligned}\text{\#params} &= (3*1000*1000)*50 + 50 \\ &= 150,000,050\end{aligned}$$

Shape of an image = (C, H, W) where C = #channels, H = height and W = width

- Huge number of parameters is difficult to optimize and easy to overfit.
- Operates on whole image: noisy background pattern, difficult to learn the pattern.

Motivation for Convolutional Layer

- Explicitly handles spatial data
- Basic computational unit is a **filter** or **kernel**:
 - **Local** connectivity (less diverse visual pattern, easier to train)
 - **Parameter sharing** (one filter, all locations)
 - **Small** size (fewer parameter, less overfitting)
- The number of parameters depends on the **filter size**, **number of filters** and the **input channels** but is **independent of input resolution**.



Input images:

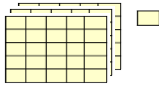
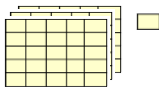


(3,32,32)

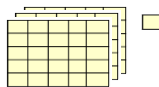


(3,1000,1000)

Filters



...



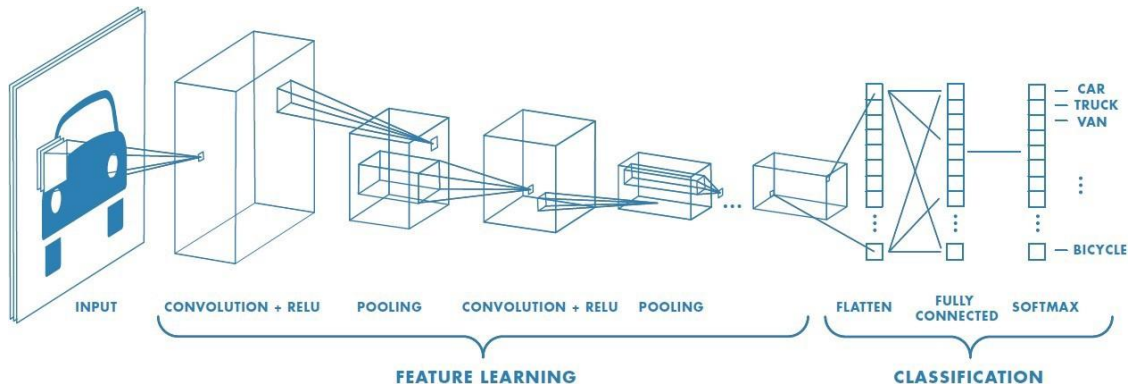
For both images:

50 x (3, 5, 5) filters

$$\begin{aligned}\text{\#params} &= 50 \cdot (3 \cdot 5 \cdot 5) + 50 \\ &= 3800\end{aligned}$$

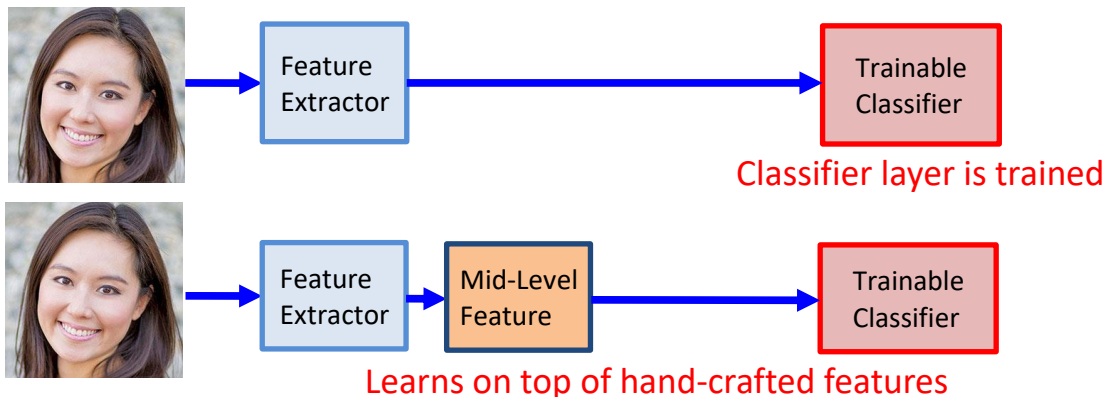
Convolutional Neural Networks

- Convolutional Neural Network (CNN or ConvNet) are created by stacking layers.
- Basic layers
 - **Convolution** (CONV) + **Activation** (e.g., ReLU) layer
 - **Pooling** layer (POOL)
 - **Fully Connected** layer (FC)
- For example:

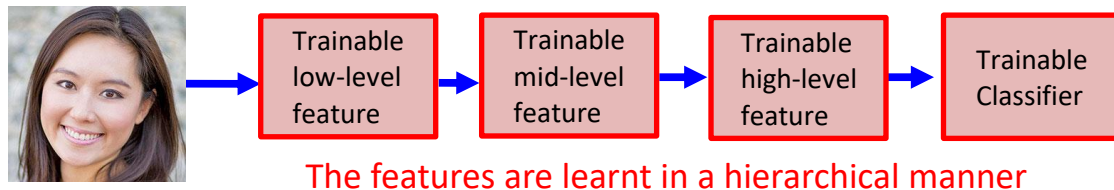


Traditional machine learning vs Deep Learning

- Traditional machine learning: Handcrafted feature extractor.

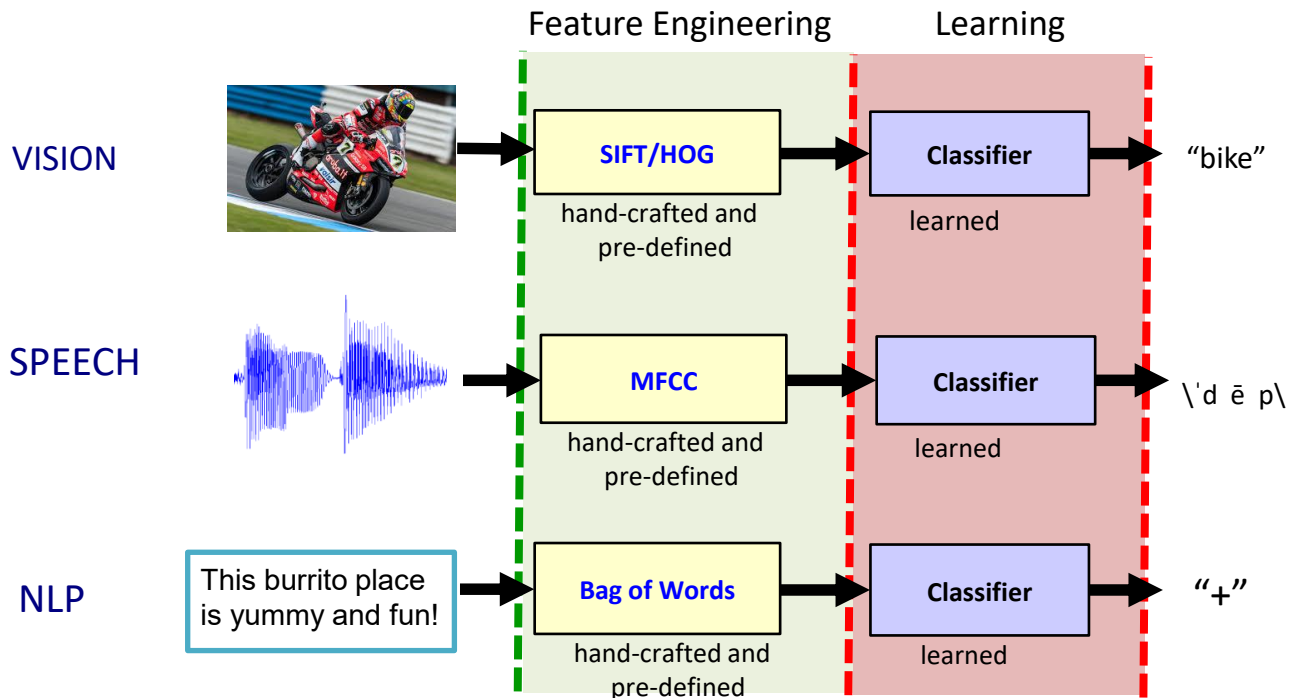


- Deep Learning: Representations are hierarchical and trained.



Traditional machine learning- Feature engineering

- In traditional Machine Learning, learning is **shallow** and the features are **hand-crafted** through **feature engineering**.



Deep Learning = Hierarchical Representation Learning

- The convolutional layers **extract spatial features** (tied to a particular spatial location).
- Deep learning **learns** representations or features **hierarchically**. The features are not hand-crafted.
 - **Higher** layer generates **higher level features**.
 - **Higher** layer features are more **class specific**.

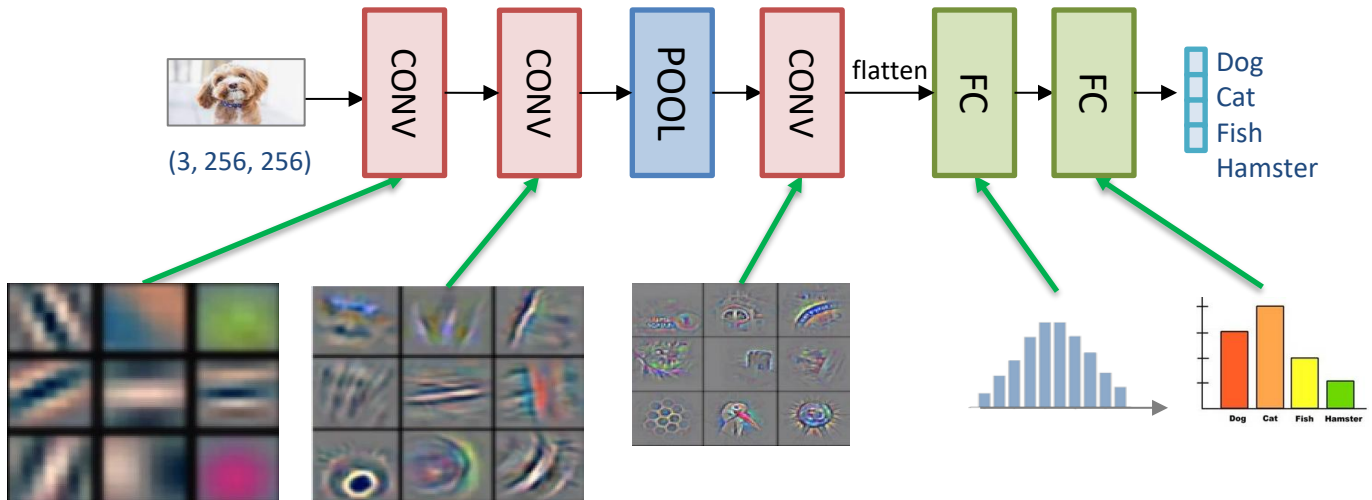
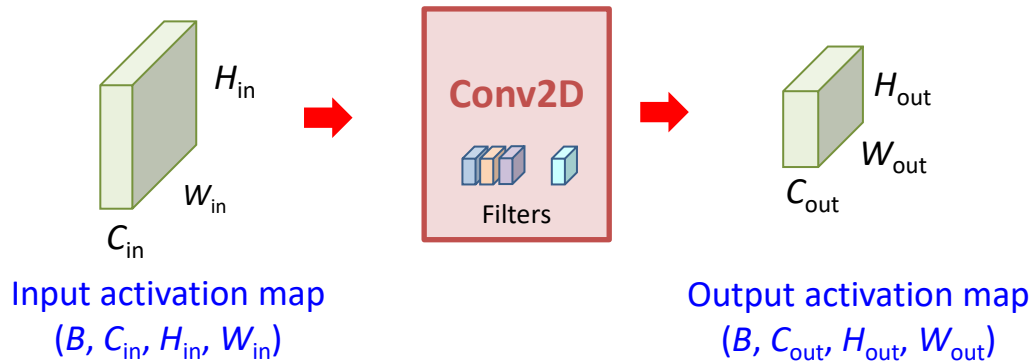


Table of Contents

1. Background
2. Convolutional Layer
3. Configuring a Convolutional Layer
4. Number of Parameters and Computations
5. Pooling Layer
6. Variants of Convolutional Layer

2D Convolution (Conv2D)

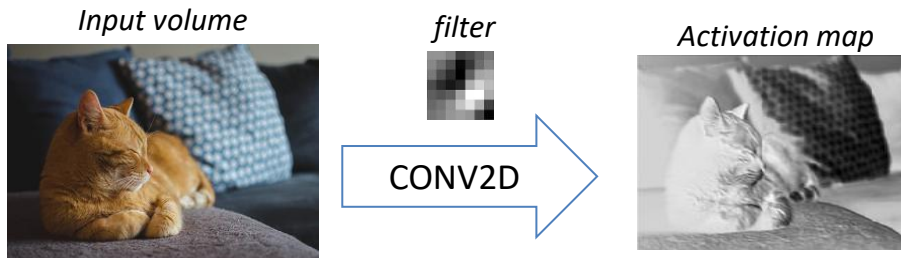
- The input and output of Conv2D layer are **spatial** data volume of shape $\text{batch_size} \times \text{num_channel} \times \text{height} \times \text{width}$ denoted as (B, C, H, W)



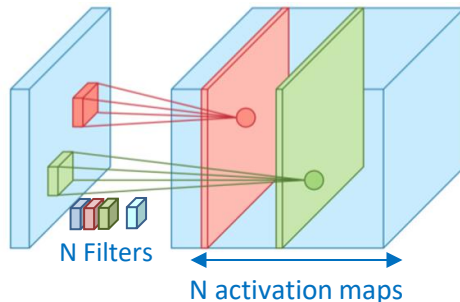
- The Conv2D layer has a set of **filters** (or **kernels**).
- Each filter is trained to extract one spatial **feature** (or **activation map**)

Activation Map

- Sliding a filter over all spatial location generates an *activation map* (or *feature map* or *response map*).



- N filters generate N activation maps



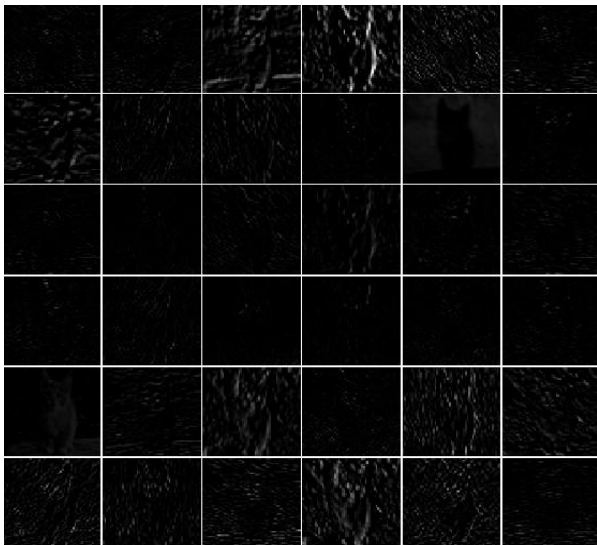
How does a filter look like?

- The figure below shows **filters** from the 1st CONV layer of a trained AlexNet. Notes: The filters have 3 channels and can be visualized directly.

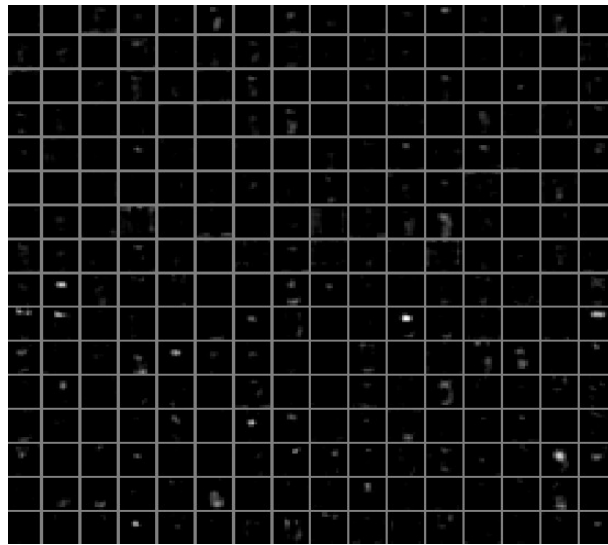


- Each filter is detecting certain kinds of visual patterns. The 1st convolutional layer are mainly extracting low level features.

How does an activation map look like?



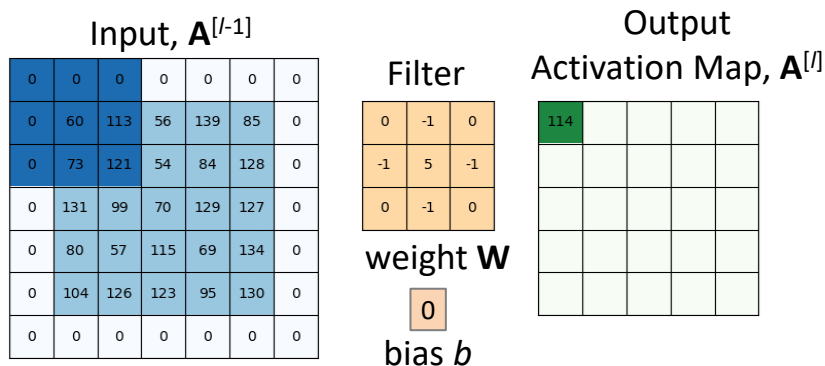
Activations map of CONV1 in AlexNet. The activations at **lower levels** are more **diffused**. The features are more low level (e.g., horizontal line features) and are pervasive throughout the image.



Activations maps of CONV5 in AlexNet. The activations map at **higher layers** are more **focused**. The features are more high level (e.g., human part features) and more localized in the image.

Single channel Conv2D Operation

- Slide the filter over all spatial locations to get filter responses (extract features from image) to get the output **activation map** or **feature map**.

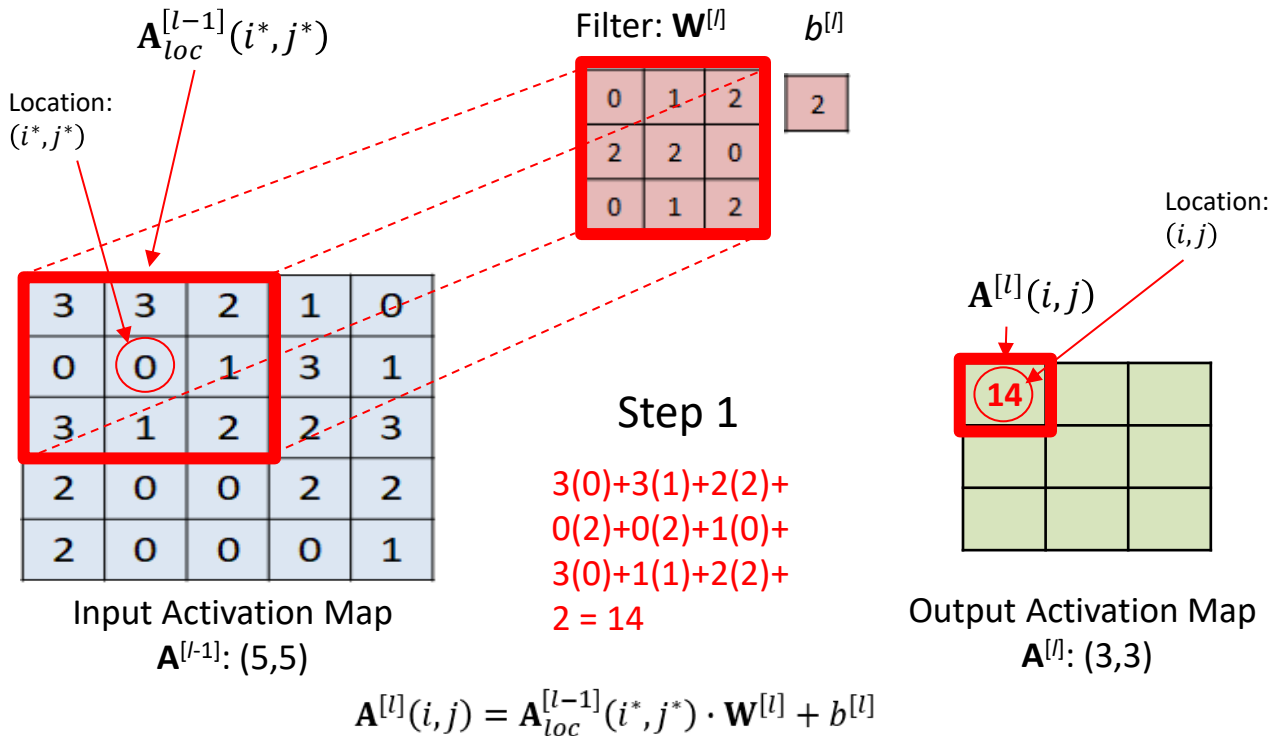


The response $\mathbf{A}^{[l]}(i, j)$ at location (i, j) is given by the **dot product** of filter $\mathbf{W}$ and the corresponding input region $\mathbf{A}_{loc}^{[l-1]}(i^*, j^*)$ and adding with the bias b .

$$\mathbf{A}^{[l]}(i, j) = \mathbf{A}_{loc}^{[l-1]}(i^*, j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$$

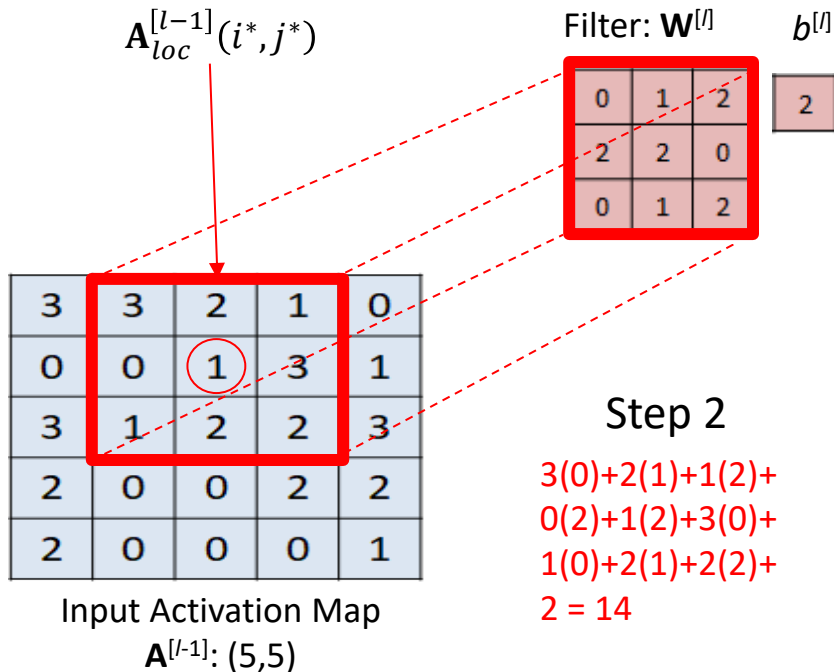
Example 1-channel Conv2D (1/10)

- Convolve a (5,5) input volume with (3,3) filter

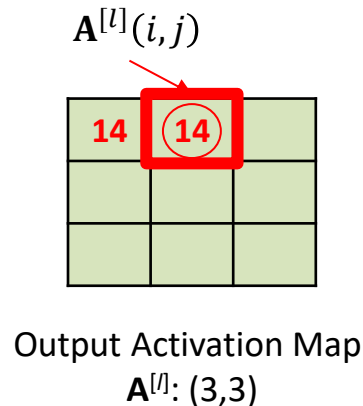


Example 1-channel Conv2D (2/10)

- Convolve a (5,5) input volume with (3,3) filter



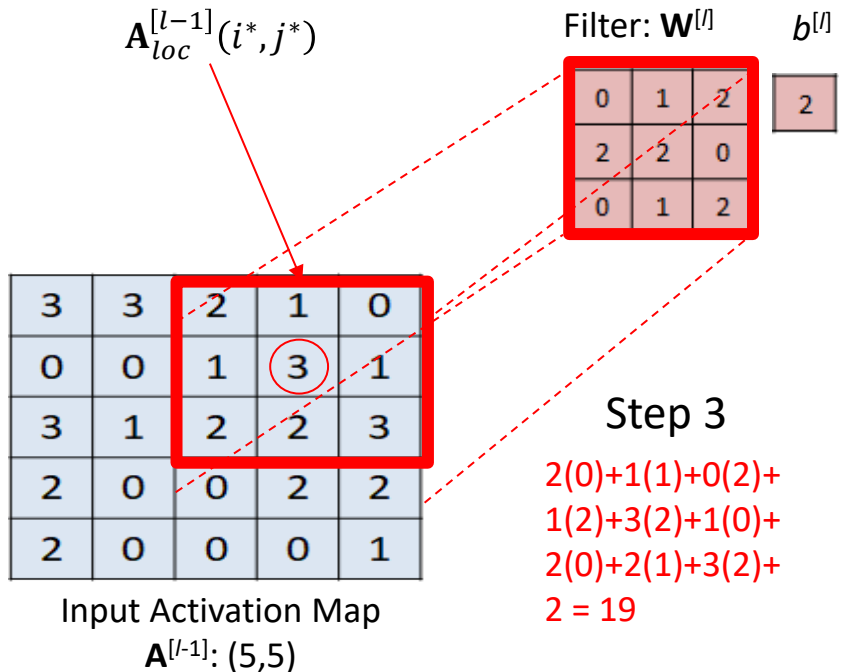
Move one step
at a time:
Stride = 1



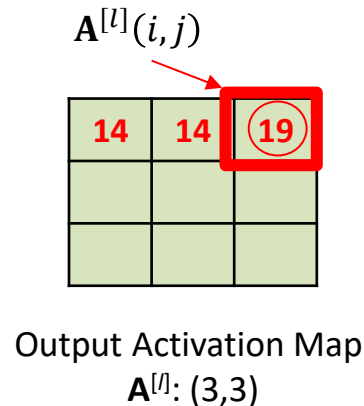
$$A^{[l]}(i, j) = A_{loc}^{[l-1]}(i^*, j^*) \cdot W^{[l]} + b^{[l]}$$

Example 1-channel Conv2D (3/10)

- Convolve a (5,5) input volume with (3,3) filter



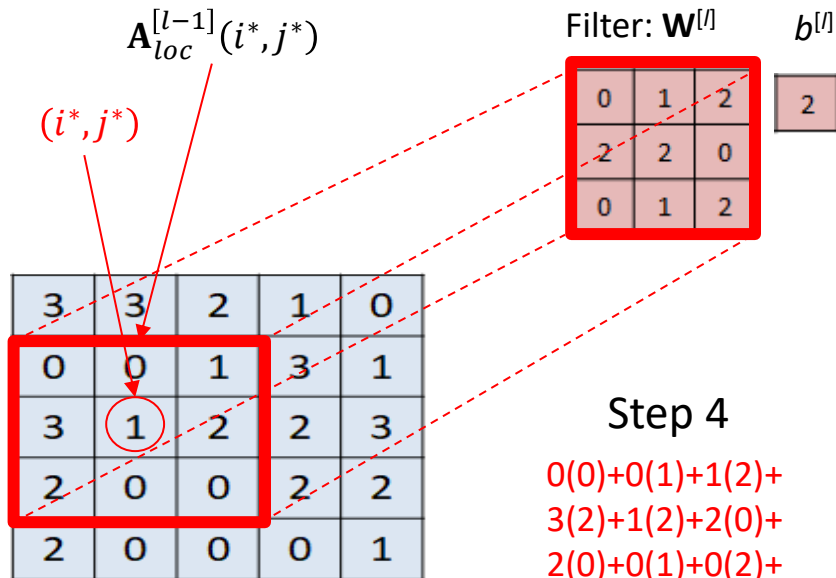
Move one step at a time:
Stride = 1



$$A^{[l]}(i, j) = A_{loc}^{[l-1]}(i^*, j^*) \cdot W^{[l]} + b^{[l]}$$

Example 1-channel Conv2D (4/10)

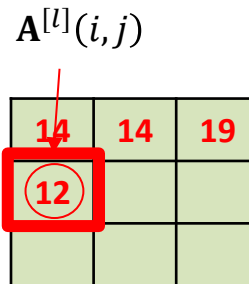
- Convolve a (5,5) input volume with (3,3) filter



Step 4

$$\begin{aligned} &0(0)+0(1)+1(2)+ \\ &3(2)+1(2)+2(0)+ \\ &2(0)+0(1)+0(2)+ \\ &2 = 12 \end{aligned}$$

Move one step
at a time:
Stride = 1

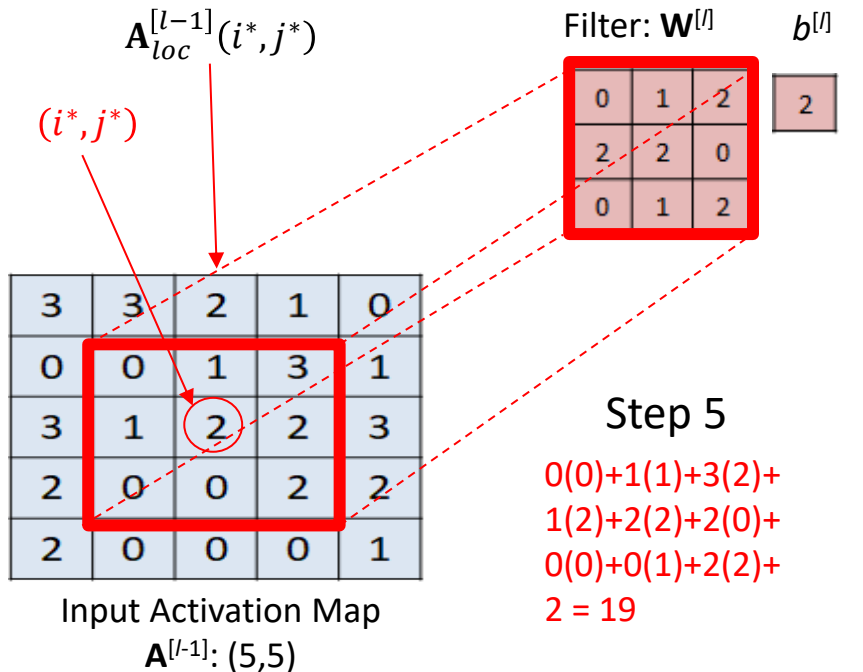


Output Activation Map
 $A^{[l]}: (3,3)$

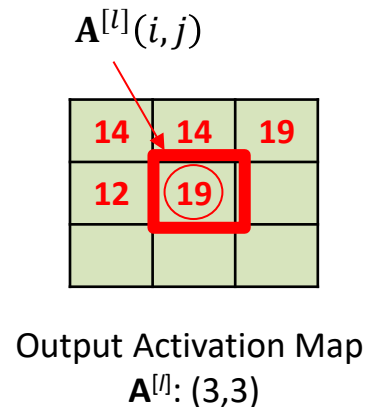
$$A^{[l]}(i,j) = A_{loc}^{[l-1]}(i^*,j^*) \cdot W^{[l]} + b^{[l]}$$

Example 1-channel Conv2D (5/10)

- Convolve a (5,5) input volume with (3,3) filter



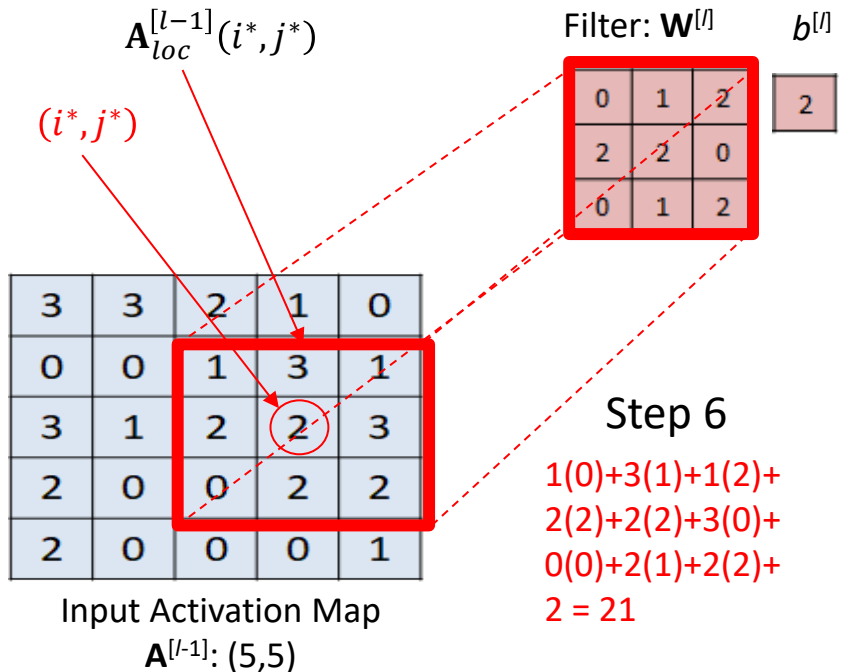
Move one step at a time:
Stride = 1



$$\mathbf{A}^{[l]}(i, j) = \mathbf{A}_{loc}^{[l-1]}(i^*, j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$$

Example 1-channel Conv2D (6/10)

- Convolve a (5,5) input volume with (3,3) filter

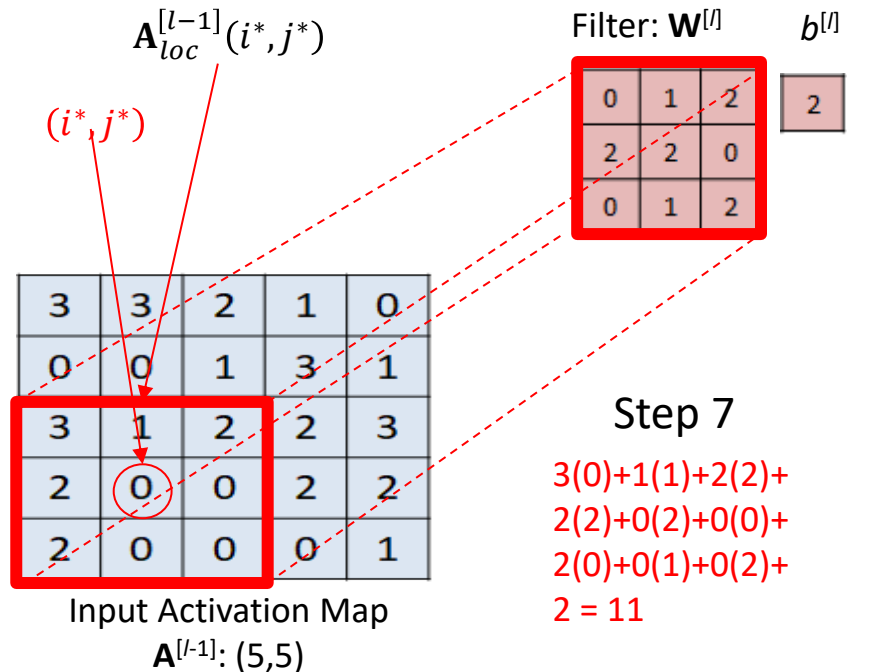


Move one step
at a time:
Stride = 1

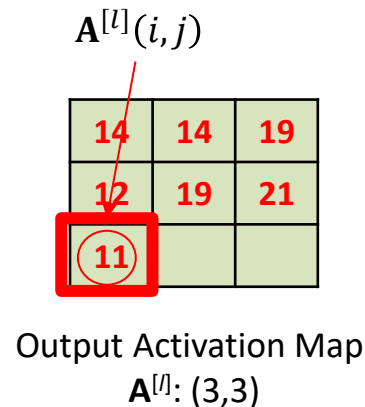
$$\mathbf{A}^{[l]}(i, j) = \mathbf{A}_{loc}^{[l-1]}(i^*, j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$$

Example 1-channel Conv2D (7/10)

- Convolve a (5,5) input volume with (3,3) filter



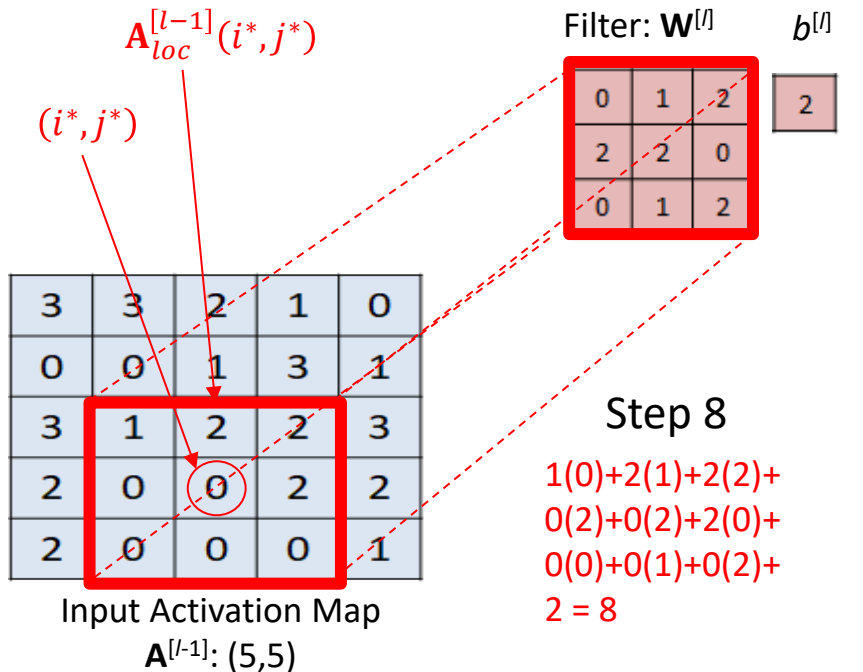
Move one step
at a time:
Stride = 1



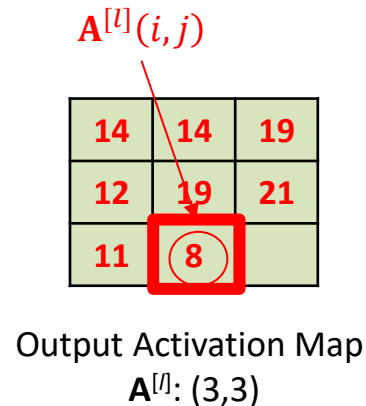
$$\mathbf{A}^{[l]}(i, j) = \mathbf{A}_{loc}^{[l-1]}(i^*, j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$$

Example 1-channel Conv2D (8/10)

- Convolve a (5,5) input volume with (3,3) filter



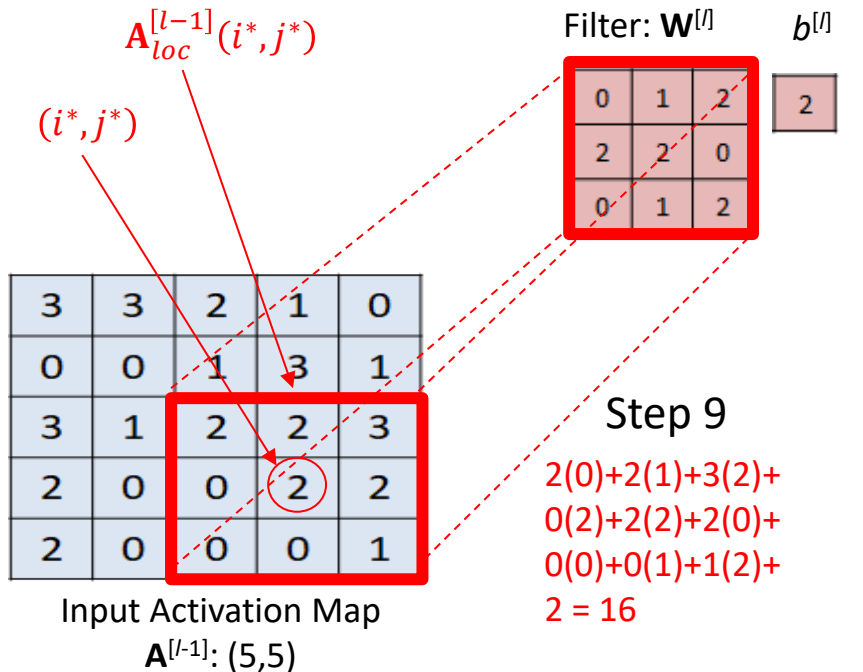
Move one step
at a time:
Stride = 1



$$\mathbf{A}^{[l]}(i, j) = \mathbf{A}_{loc}^{[l-1]}(i^*, j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$$

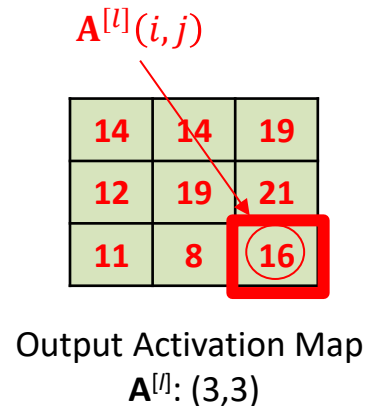
Example 1-channel Conv2D (9/10)

- Convolve a (5,5) input volume with (3,3) filter



$$\mathbf{A}^{[l]}(i, j) = \mathbf{A}_{loc}^{[l-1]}(i^*, j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$$

Move one step
at a time:
Stride = 1



Example 1-channel Conv2D (10/10)

- Convolve a (5,5) input volume with (3,3) filter

Filter: $\mathbf{W}^{[l]}$ $b^{[l]}$

0	1	2
2	2	0
0	1	2

2

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

Input Activation Map
 $\mathbf{A}^{[l-1]}: (5,5)$

Convolve the input with
the (3,3) filter, $s = 1$



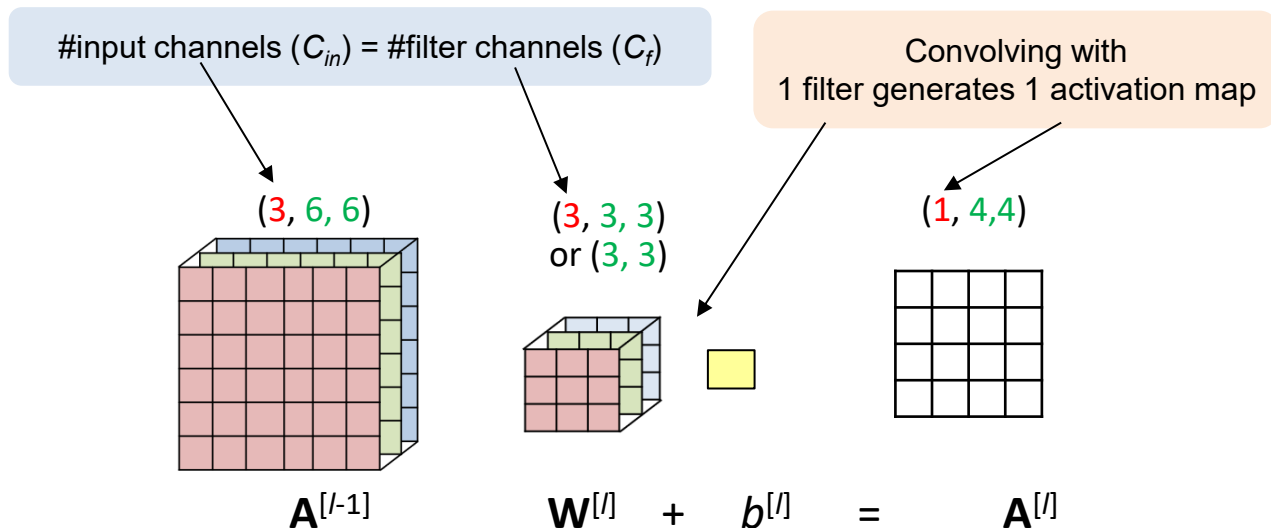
14	14	19
12	19	21
11	8	16

Output Activation Map
 $\mathbf{A}^{[l]}: (3,3)$

$$\mathbf{A}^{[l]}(i,j) = \mathbf{A}_{loc}^{[l-1]}(i^*, j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$$

Multi-channel Conv2D Operation

- The input volume may have **multiple channels**. For 2D convolutions, the filters have the **same number of channels** as the input volume.



Notes: The dimension of filter is often written in terms of the spatial dimension only (3, 3). In this case, C_f can be inferred from C_{in}

Example multi-channel Conv2D Operation

- Simulation for multi-channel Conv2D operation

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

+

+

↑
Bias = 1

+ 1 = -25

Output

-25				...
				...
				...
				...
...	...	...	...	...

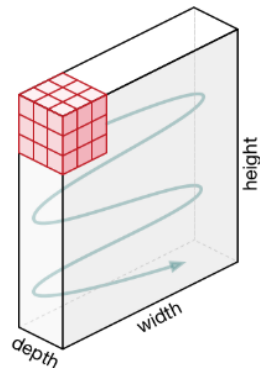


Table of Contents

1. Background
2. Convolutional Layer
3. Configuring a Convolutional Layer
4. Number of Parameters and Computations
5. Pooling Layer
6. Variants of Convolutional Layer

Coding 2D Convolution in PyTorch

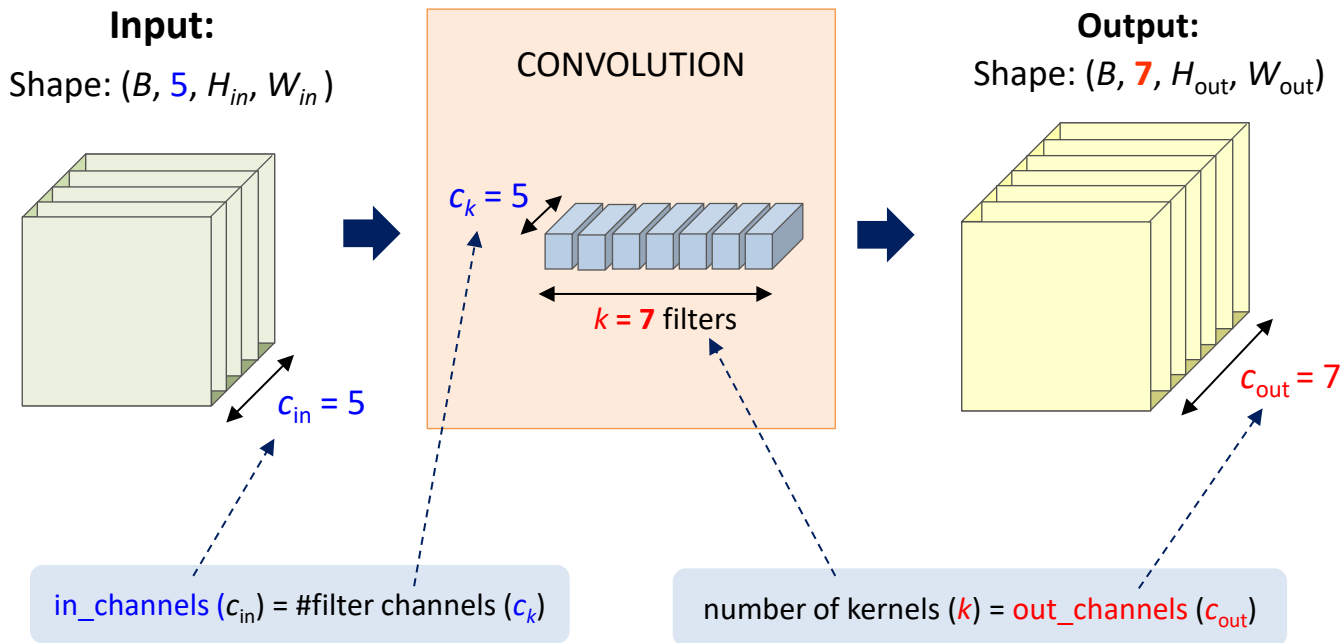


```
torch.nn.Conv2d (in_channels, out_channels, kernel_size,  
stride=1, padding=0, ...)
```

- Create a 2D convolution layer
- Arguments:
 - `in_channels`: number of channels in the input tensor (c_{in}).
 - `out_channels`: number of channels in the output tensor (c_{out}).
 - `kernel_size`: spatial dimension of the filter (f).
 - `stride`: stride for the cross-correlation (s).
 - `padding`: padding to be applied to the input (p).

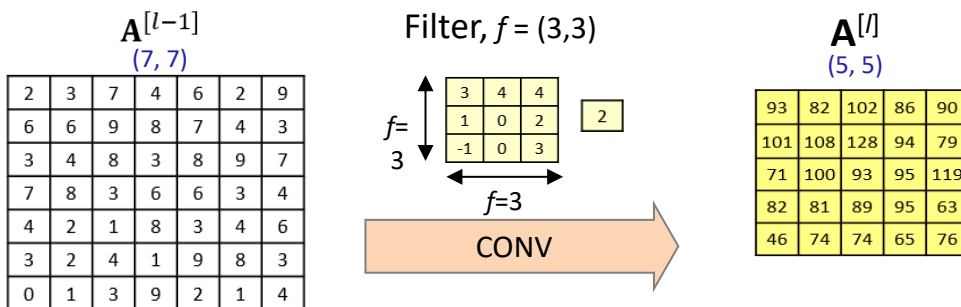
Hyperparameter setting: (1) in_channels and (2) out_channels

- **in_channels** is the number of channels in the input volume.
- **out_channels** is the number of channels in the output volume.



Hyperparameter setting: (3) Kernel size f

- Kernel size or f refers to the size of the filter. Typically set to 3.



Smaller kernel size ($f = 3$)	Bigger kernel size ($f \geq 5$)
Smaller receptive field: Better at capturing small details and local patterns.	Larger receptive field: better at capturing global info. Less fine detail, may miss subtle patterns.
Deeper network required to improve receptive field. But more non-linearity makes the network more expressive .	Shallower network is sufficient to needed to larger features. But less non-linearity makes the network less expressive .
Less parameter: less overfitting, less memory usage faster computation.	More parameters: more prone to overfitting, more memory usage and longer computation per layer.

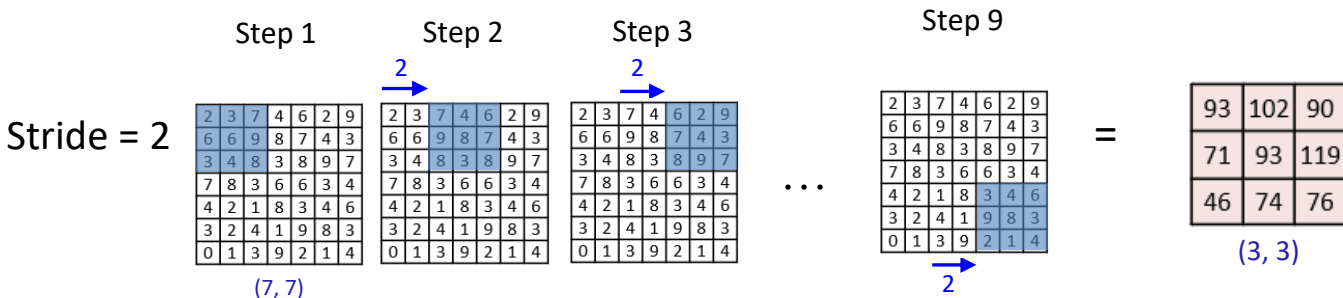
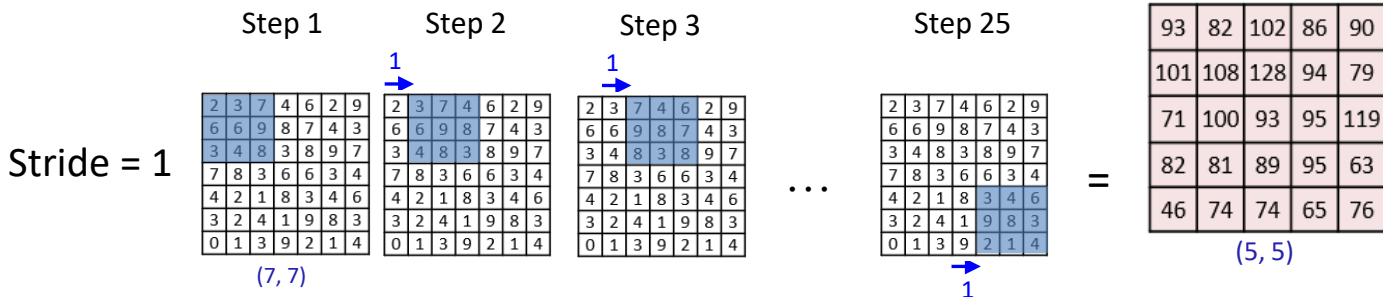
Hyperparameter setting: (3) Kernel size f

- Common practice:

Size	Usage
3x3	Widely used in modern CNN architectures (like VGG and ResNet) because they balance feature extraction, efficiency, and expressiveness.
1x1	Useful for dimensionality reduction and creating complex interactions between channels (like in Inception networks).
5x5, 7x7	Used in the first convolutional layer for some network to capture broader features from raw data.

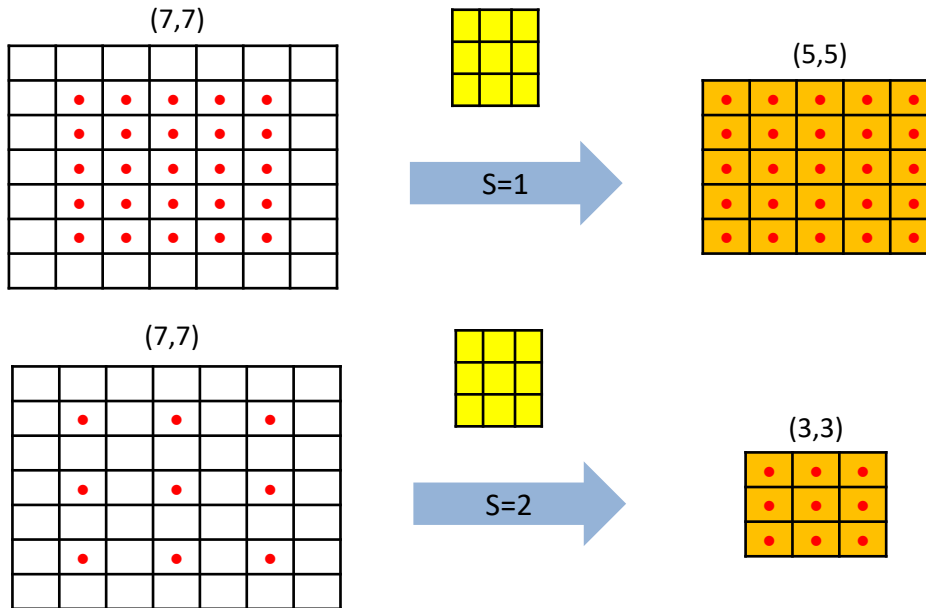
Hyperparameter setting: (4) Stride s

- **Stride** is the number of steps by which the filter moves.
- Determines the resolution of the output activation map. Using a larger stride shrinks the activation map. When $s=2$, resolution is **halved**.



Hyperparameter setting: (4) Stride s

- Size of the activation map = Number of local regions being compared with the filter



Hyperparameter setting: (4) Stride s

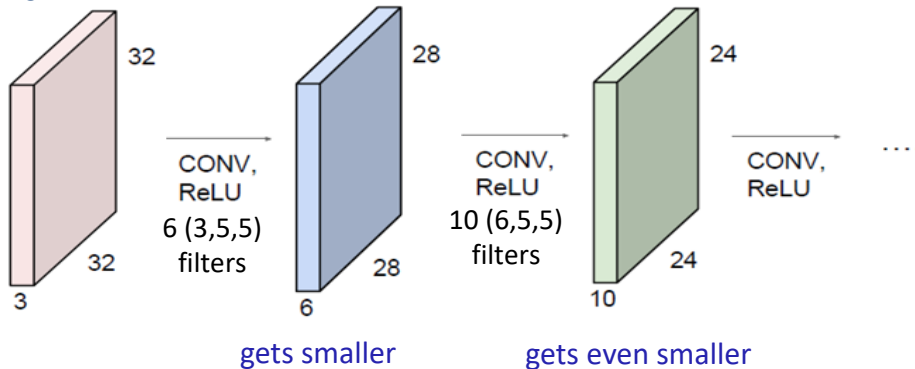
- Smaller vs bigger stride

Smaller stride ($s = 1$)	Bigger stride ($s > 2$)
Activation map is closer to input resolution.	Reduces the output resolution.
More positions sampled from input. Captures finer features with overlapping receptive fields.	Sparser positions sampled from input. Captures coarser feature map. May miss smaller or localized patterns due to skipped positions.
Higher computations due to more convolution operations.	Fewer computations due to more convolution operations.
More memory due to larger activation map generated.	Less memory due to smaller activation map generated.

Hyperparameter setting: (5) Padding p

- **Issue:** The resolution **reduce** with each layer

$f=5, s=1, p=0$

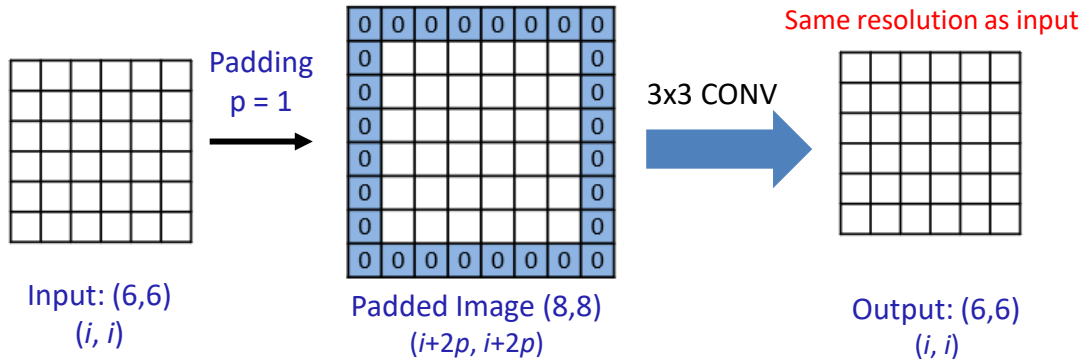


(32,32) input convolved repeatedly with a (5,5) filter shrinks volumes spatially! (32 $\rightarrow$ 28 $\rightarrow$ 24 ...)

- For $i = 32$ and $f = 5$, network cannot be deeper than 8.
- If spatial resolution shrinks too fast, cannot go very deep.

Hyperparameter setting: (5) Padding p

- Padding inserts additional boundary rows or columns to the input.



- Used to **preserve** the spatial resolution
- Prevents information loss by keeping information at the **border**.
Otherwise, border pixels will be under-represented in the output (wash away effect).

Hyperparameter setting: (5) Padding p

Types of padding:

- **Valid Padding:**
No padding performed. Feature map size shrinks after each convolution.
- **Same Padding:**
Pads input so output feature map size matches the input size.
- **Custom Padding:**
Adds a specific number of pixels based on design choice.

Hyperparameter setting: (5) Padding p

Padding values

Zero Padding	Mirror Padding	Replicate Padding																																																																																																																																																																																																
Fills the border with 0 <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>0</td></tr><tr><td>0</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>0</td></tr><tr><td>0</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>0</td></tr><tr><td>0</td><td>7</td><td>8</td><td>9</td><td>10</td><td>11</td><td>12</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	0	0	0	0	0	1	2	3	4	5	6	0	0	7	8	9	10	11	12	0	0	1	2	3	4	5	6	0	0	7	8	9	10	11	12	0	0	1	2	3	4	5	6	0	0	7	8	9	10	11	12	0	0	0	0	0	0	0	0	0	Mirrors edge values <table><tr><td>11</td><td>10</td><td>9</td><td>10</td><td>11</td><td>12</td><td>11</td><td>10</td></tr><tr><td>7</td><td>6</td><td>5</td><td>6</td><td>7</td><td>8</td><td>7</td><td>6</td></tr><tr><td>3</td><td>2</td><td>1</td><td>2</td><td>3</td><td>4</td><td>3</td><td>2</td></tr><tr><td>7</td><td>6</td><td>5</td><td>6</td><td>7</td><td>8</td><td>7</td><td>6</td></tr><tr><td>11</td><td>10</td><td>9</td><td>10</td><td>11</td><td>12</td><td>11</td><td>10</td></tr><tr><td>15</td><td>14</td><td>13</td><td>14</td><td>15</td><td>16</td><td>15</td><td>14</td></tr><tr><td>11</td><td>10</td><td>9</td><td>10</td><td>11</td><td>12</td><td>11</td><td>10</td></tr><tr><td>7</td><td>6</td><td>5</td><td>6</td><td>7</td><td>8</td><td>7</td><td>6</td></tr></table>	11	10	9	10	11	12	11	10	7	6	5	6	7	8	7	6	3	2	1	2	3	4	3	2	7	6	5	6	7	8	7	6	11	10	9	10	11	12	11	10	15	14	13	14	15	16	15	14	11	10	9	10	11	12	11	10	7	6	5	6	7	8	7	6	Repeats edge values <table><tr><td>1</td><td>1</td><td>1</td><td>2</td><td>3</td><td>4</td><td>4</td><td>4</td></tr><tr><td>1</td><td>1</td><td>1</td><td>2</td><td>3</td><td>4</td><td>4</td><td>4</td></tr><tr><td>1</td><td>1</td><td>1</td><td>2</td><td>3</td><td>4</td><td>4</td><td>4</td></tr><tr><td>5</td><td>5</td><td>5</td><td>6</td><td>7</td><td>8</td><td>8</td><td>8</td></tr><tr><td>1</td><td>1</td><td>1</td><td>2</td><td>3</td><td>4</td><td>4</td><td>4</td></tr><tr><td>5</td><td>5</td><td>5</td><td>6</td><td>7</td><td>8</td><td>8</td><td>8</td></tr><tr><td>5</td><td>5</td><td>5</td><td>6</td><td>7</td><td>8</td><td>8</td><td>8</td></tr><tr><td>5</td><td>5</td><td>5</td><td>6</td><td>7</td><td>8</td><td>8</td><td>8</td></tr></table>	1	1	1	2	3	4	4	4	1	1	1	2	3	4	4	4	1	1	1	2	3	4	4	4	5	5	5	6	7	8	8	8	1	1	1	2	3	4	4	4	5	5	5	6	7	8	8	8	5	5	5	6	7	8	8	8	5	5	5	6	7	8	8	8
0	0	0	0	0	0	0	0																																																																																																																																																																																											
0	1	2	3	4	5	6	0																																																																																																																																																																																											
0	7	8	9	10	11	12	0																																																																																																																																																																																											
0	1	2	3	4	5	6	0																																																																																																																																																																																											
0	7	8	9	10	11	12	0																																																																																																																																																																																											
0	1	2	3	4	5	6	0																																																																																																																																																																																											
0	7	8	9	10	11	12	0																																																																																																																																																																																											
0	0	0	0	0	0	0	0																																																																																																																																																																																											
11	10	9	10	11	12	11	10																																																																																																																																																																																											
7	6	5	6	7	8	7	6																																																																																																																																																																																											
3	2	1	2	3	4	3	2																																																																																																																																																																																											
7	6	5	6	7	8	7	6																																																																																																																																																																																											
11	10	9	10	11	12	11	10																																																																																																																																																																																											
15	14	13	14	15	16	15	14																																																																																																																																																																																											
11	10	9	10	11	12	11	10																																																																																																																																																																																											
7	6	5	6	7	8	7	6																																																																																																																																																																																											
1	1	1	2	3	4	4	4																																																																																																																																																																																											
1	1	1	2	3	4	4	4																																																																																																																																																																																											
1	1	1	2	3	4	4	4																																																																																																																																																																																											
5	5	5	6	7	8	8	8																																																																																																																																																																																											
1	1	1	2	3	4	4	4																																																																																																																																																																																											
5	5	5	6	7	8	8	8																																																																																																																																																																																											
5	5	5	6	7	8	8	8																																																																																																																																																																																											
5	5	5	6	7	8	8	8																																																																																																																																																																																											
Simple and efficient, prevents information loss.	Reduces edge distortion and preserves local structure.	Preserves edge information, simple implementation																																																																																																																																																																																																
May introduce artificial edges, may distort patterns.	Can create repeated patterns near the borders.	Can exaggerate border features, may cause artifacts																																																																																																																																																																																																
Common in most standard CNNs (e.g., ResNet) to preserve spatial dimensions and avoid feature loss.	Used for segmentation task UNet (UNet by Ronneberger et al., 2015) and denoising autoencoders.	Used for style transfer (e.g., Gatys et al., 2016) and texture synthesis.																																																																																																																																																																																																

Notes: No hard criteria on which type of padding to use. It depends on the application.

Formula to compute output resolution

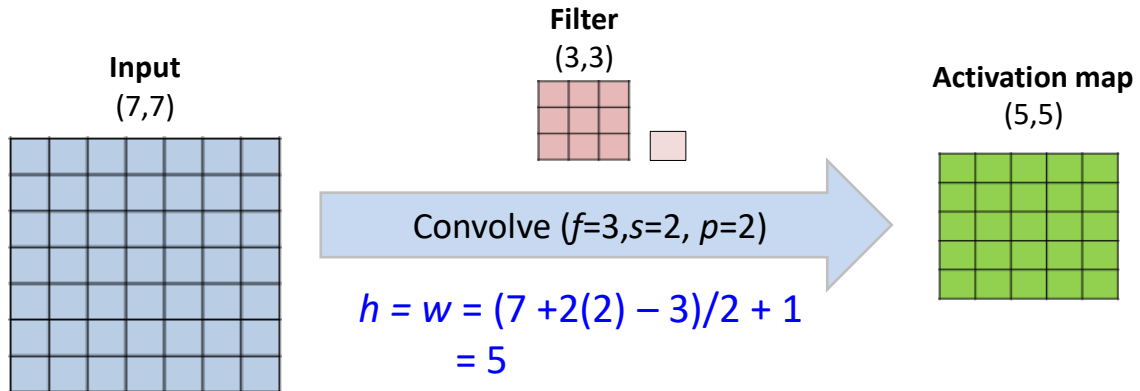
- Given image size i , filter size f and stride s :

Size of activation map:

$$o = \left\lfloor \frac{i + 2p - f}{s} + 1 \right\rfloor$$

Round down: do not perform dot product if local region is out of scope

- For example ($i = 7, p = 2, f = 3, s = 2$):



Exercise

Given the following input volume with shape (2, 4, 4), and CONV2D layer:

Input
volume:

$\mathbf{A}^{[l-1]} = \text{array}(\begin{bmatrix} [1, 4, 5, 1], \\ [3, 3, 5, 4], \\ [2, 5, 1, 5], \\ [3, 3, 3, 4], \\ [2, 4, 4, 5], \\ [2, 5, 3, 5], \\ [1, 1, 3, 4], \\ [1, 1, 2, 4] \end{bmatrix})$

filter:

$\mathbf{W}^{[l]} = \text{array}(\begin{bmatrix} [0, 1], \\ [0, 0], \\ [0, 1], \\ [1, 0] \end{bmatrix})$ $b^{[l]} = 2$

- What is the shape of the output volume when $p=0, s=2$?

$$o = (i + 2p - f)/s + 1 = (4 + 0 - 2)/2 + 1 = 2$$

Shape of output volume = (1, 2, 2)

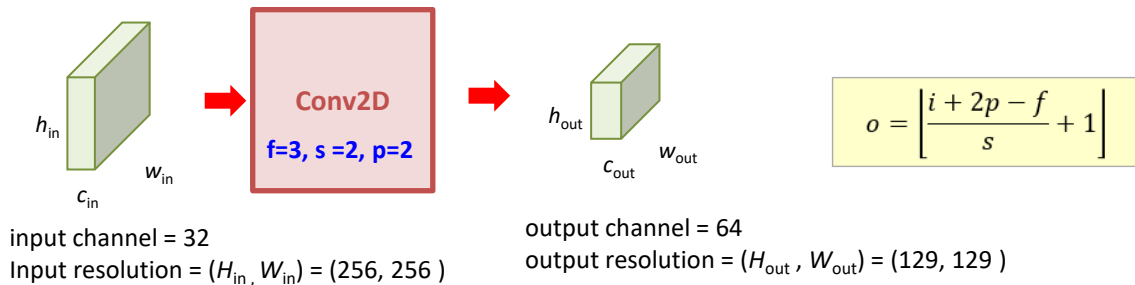
- What is the resultant activation map?

12	11
9	13

Configuring the CONV layer

- Use small **filter**, e.g., (3,3)
- For most cases, use stride of 1:
 - In general, smaller strides work better in practice
 - Leaves all spatial down-sampling to the POOL layers while CONV only transform the input volume depth-wise
- Use **stride** of 1 with **padding** to maintain spatial dimensions

Implementing a Convolutional Layer in PyTorch



Implementing a convolutional layer

```
>>> import torch.nn as nn
>>> import torch

# create a convolutional layer
>>> conv1 = nn.Conv2d(in_channels=32, out_channels=64,
                      kernel_size=3, stride=2, padding=2)

>>> x = torch.randn(4, 32, 256, 256) # create dummy input, batch size=4

# forward propagation on conv1.
# Note that we have not trained the layer yet.
>>> y = conv1(x)
```

Input: Tensor of shape $(B, C_{in}, H_{in}, W_{in})$
Output: Tensor of shape $(B, C_{out}, H_{out}, W_{out})$

Table of Contents

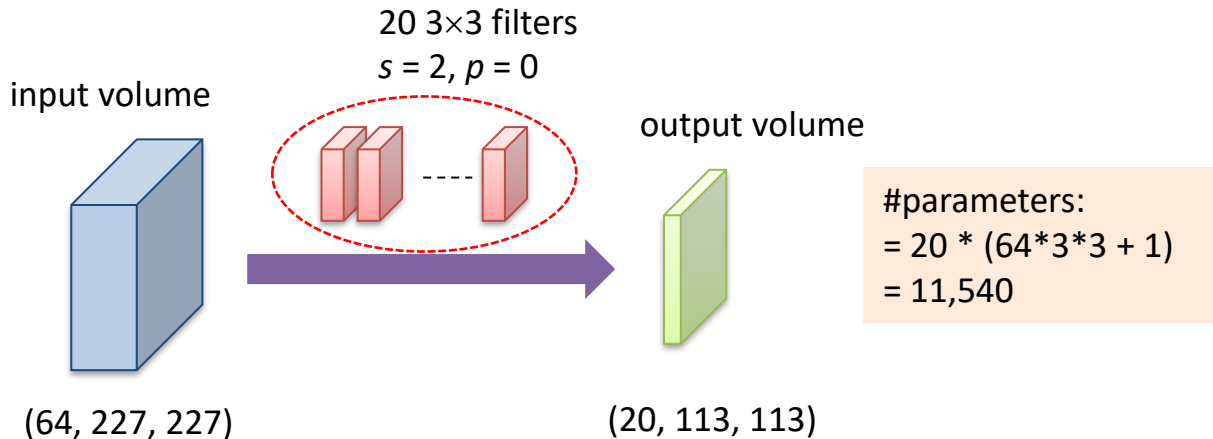
1. Background
2. Convolutional Layer
3. Configuring a Convolutional Layer
4. Number of Parameters and Computations
5. Pooling Layer
6. Variants of Convolutional Layer

Number of parameters in convolutional layers

- Number of parameters in a convolutional layer is given by:

$$\text{\#parameters} = \text{\#filter} \times \text{\#params per filter}$$

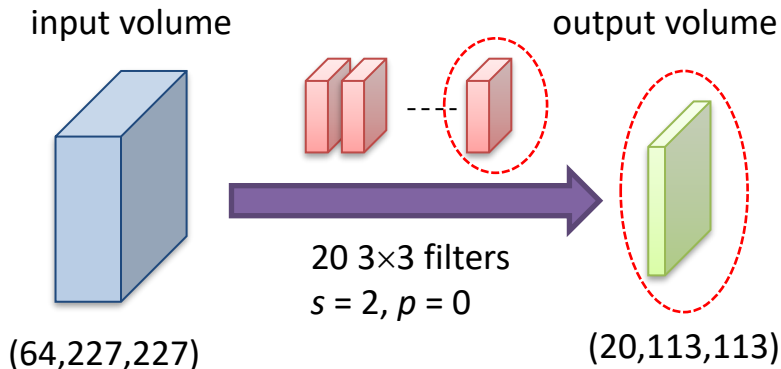
Example:



Number of computations in convolutional layers

- Number of computations required to generate the output layer is given by:

$$\begin{aligned}\text{\#computations} &= \text{\#output activations} \times \text{\#computations per activation} \\ &= \text{output volume size} \times \text{\#params per filter}\end{aligned}$$



$$\begin{aligned}\text{\#Computations:} \\ &= (20 \times 113 \times 113) \times (64 \times 3 \times 3 + 1) \\ &= 147,354,260\end{aligned}$$

Example

Given input volume shape = (5, 200,200), 10 (3,3) filters with stride 1, and padding 1:

What is the output volume size?

$$o = (200 - 3 + 2*1)/1 + 1 = 200$$

$$\text{Output volume size} = (10, 200, 200)$$

What is the number of parameters in this layer?

$$\text{Each filter has } 5*3*3 + 1 = 46 \text{ parameters}$$

$$\text{param_num} = 10 * 46 = 460$$

What is the number of computations required for this layer?

$$\begin{aligned} \# \text{ computation} &= \text{activations_in_output} \times \text{params_in_filter} \\ &= (200 \times 200 \times 10) \times 46 = 18,400,000 \end{aligned}$$

Table of Contents

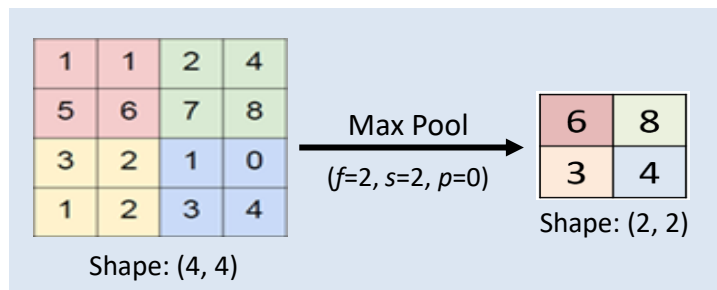
1. Background
2. Convolutional Layer
3. Configuring a Convolutional Layer
4. Number of Parameters and Computations
5. Pooling Layers
6. Variants of Convolutional Layer

Pooling layer

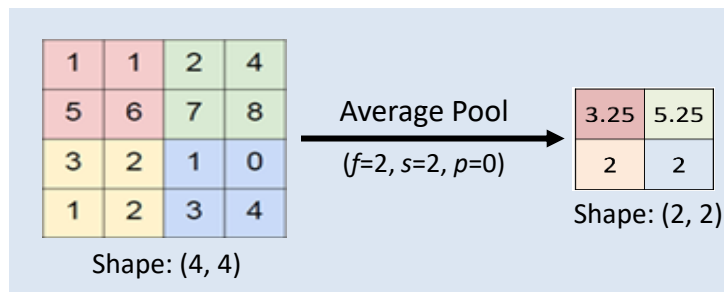
- Similar to CONV layer, slide a filter over each channel of feature map but **summarizes** (e.g. max or average) the values in each region.

(Notes: Use $o = \left\lfloor \frac{i+2p-f}{2} + 1 \right\rfloor$ to compute the output resolution)

- Max Pooling:



- Average Pooling:

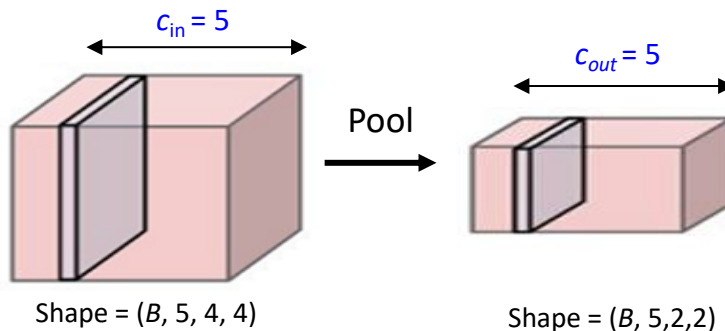


Why pooling?

1. **Smaller feature maps:** Fewer computations and memory consumption
2. **Feature generalization:** Retains essential information while discarding less important details including noise.
3. **Reduced sensitivity to noise and spatial variation:** Helps network recognize features without being too precise about their exact position. Ignores minor changes in the input. Improves generalization across different input variations.
4. **Reducing overfitting:** feature generalization and reduced noise sensitivity and translation invariance offer some regularization effect and helps reduce overfitting.

Pooling layer

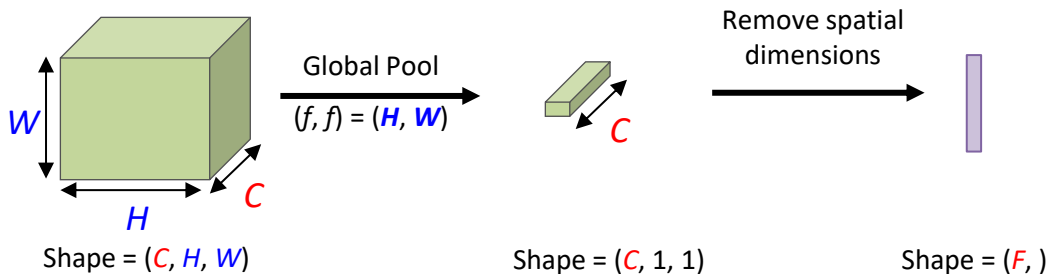
- No parameters to be learnt
- Preserves the number of channels.



- Most common setting:
 - Max pooling
 - $f = 2, s = 2$ (discards 75% of the activations in the input volume)
 - $f = 3, s = 2$
 - No padding
 - Uncommon to see max pooling with $f > 3$ (too lossy and aggressive, usually leads to worse performance)

Global Pooling

- **Global Pooling** is pooling with kernel size $(f, f) = \text{input resolution } (H, W)$
- Summarize the **whole** spatial dimension and transforms a **3D** spatial data with shape (C, H, W) into a **1D** feature vector with shape $(F,)$.



- Typically used as an **interface** between convolutional layer and linear layer.

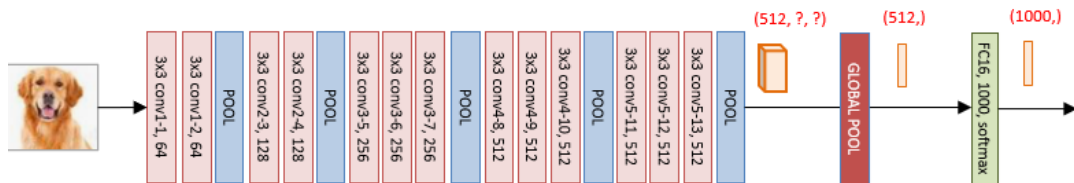
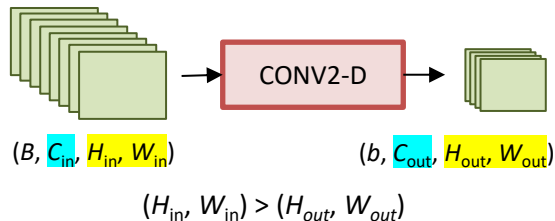


Table of Contents

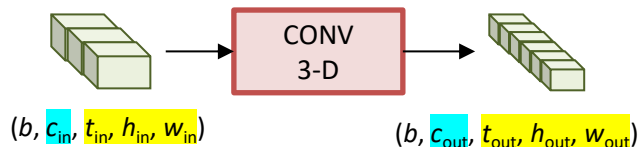
1. Background
2. Convolutional Layer
3. Configuring a Convolutional Layer
4. Number of Parameters and Computations
5. Pooling Layer
6. Variants of Convolutional Layer

Types of Convolutions

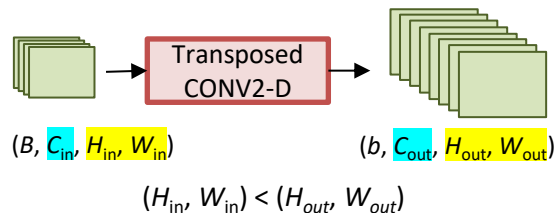
2-D Convolution:



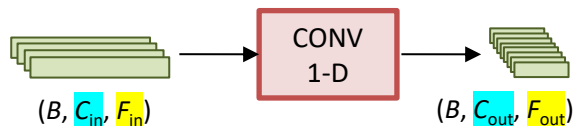
3-D Convolution:



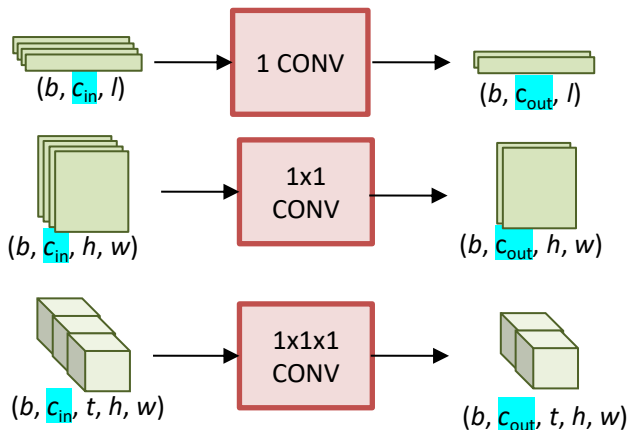
Transposed Convolution:



1-D Convolution:

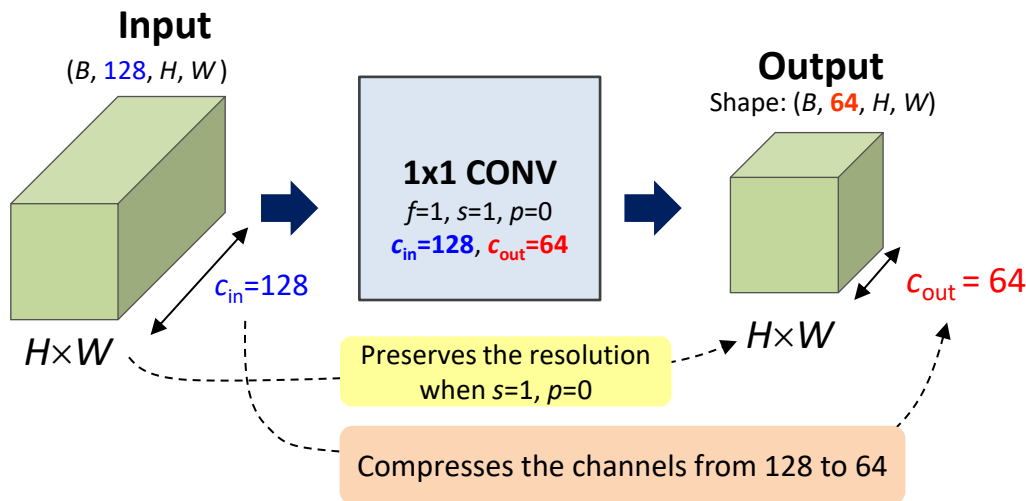


Filter size $f=1, s=1, p=0$

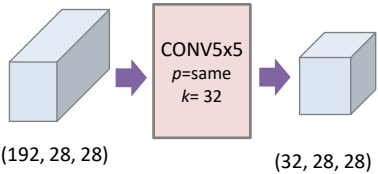
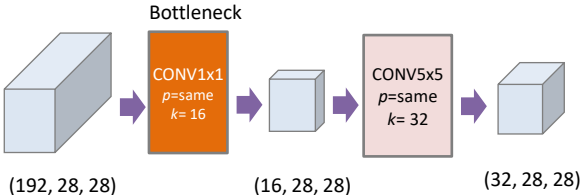


1x1 Convolution ($f = 1$)

- Convolution operation with a **kernel size of 1x1**, applied across the depth of the input. Typically used with $s = 1, p = 0$ to preserve the resolution.
- Used to **Dimensionality reduction** to reduce computation cost in subsequent 3x3 convolution operations.
 - **Restore** back the channels to its original dimensionality
 - Increases **non-linearity** when combined with activation function



Why 1x1 Convolution?

	Without bottleneck layer	With bottleneck layer
Architecture	 (192, 28, 28) → CONV5x5 $p=\text{same}$ $k=32$ → (32, 28, 28)	 (192, 28, 28) → Bottleneck CONV1x1 $p=\text{same}$ $k=16$ → (16, 28, 28) → CONV5x5 $p=\text{same}$ $k=32$ → (32, 28, 28)
Total Computations	$\text{Total} = (32 \cdot 28 \cdot 28) \cdot (5 \cdot 5 \cdot 192 + 1)$ $= 120\text{M}$	$\text{Conv1x1} = (28 \cdot 28 \cdot 16) \cdot (1 \cdot 1 \cdot 192 + 1) = 2.4\text{M}$ $\text{Conv5:5} = (28 \cdot 28 \cdot 32) \cdot (5 \cdot 5 \cdot 16 + 1) = 10\text{M}$ $\text{Total} = 12.4\text{M (reduced by a factor of 10)}$
Number of parameters	$\text{Total} = 32 \cdot (5 \cdot 5 \cdot 192 + 1)$ $= 154\text{K}$	$\text{Conv1x1} = 16 \cdot (1 \cdot 1 \cdot 192 + 1) = 3\text{K}$ $\text{Conv5:5} = 32 \cdot (5 \cdot 5 \cdot 16 + 1) = 12\text{K}$ $\text{Total} = 15\text{K (reduced by a factor of 10)}$
Expressiveness	Less expressive (only one non-linearity)	More expressive (two non-linearities)

*Thank you
for your attention*